


# BLOCK 2: Hardware & Infrastructure Efficiency

14th October 2026, Zurich




# WELCOME

## Frédéric Nass



14th October 2026, Zurich

CEPH DAYS ZURICH · OCTOBER 14, 2026

# Every Byte Counts!

## Ceph Capacity Optimization in Expensive Times

Frédéric Nass · CLYSO
Ceph Ambassador & Ceph Foundation Board Member



# Agenda

**01** **The context**
Hardware prices are soaring
Budgets shrinking

**02** **Managing RBD images right**
EC pools, UNMAP reclamation
EC performance, VDO dedup

**03** **What if I'm using CephFS?**
File layouts to an EC pool
Migrate existing data, reclaim space

**04** **S3: optimization, compression & deduplication**
Compression, dynamic placement
Optimized retention, deduplication

**05** **And at the RADOS layer?**
EC, FastEC, BlueStore compression
RocksDB compression, size=2: don't!

**06** **Hardware**
OSD memory allocation, ZRAM
Consumer drives, key takeaways

SECTION 01

# The context

Hardware prices are soaring, budgets are shrinking



# The context

- AI has sent hardware prices **soaring**
- Budgets are getting **tighter**
- Limited hardware **availability** and extended **lead times**



**Good news**: Ceph offers several levers to reduce the data footprint without sacrificing reliability. Let's see how to claw back space in our clusters!

SECTION 02

# Managing RBD images right

EC pools, UNMAP reclamation, EC performance, VDO dedup



# RBD: which pool for our images

### Replication or erasure coding?

- **Replication** for database disks, VM OS disks, and any workload that requires **< 1 ms latency** on reads/writes.
- **Erasure** coding for large disks and big files that stripe well (**k+m**) with a rather “quiet” IO profile. Multiple StorageClasses in Kubernetes.

### Raw space savings

Moving (**pvmove**) a **1 TB** RBD image from RF3 to an EC4+2 pool reclaims **1.5 TB**!



```
# New image: metadata replicated, data on EC
$ rbd create --size 1T --data-pool rbd-ec rbd/vm-disk
# Existing image: live-migrate its data to EC
$ rbd migration prepare --data-pool rbd-ec \
    rbd/vm-old rbd/vm-new
$ rbd migration execute rbd/vm-new
$ rbd migration commit rbd/vm-new
# ceph-csi StorageClass:  dataPool: rbd-ec
```

# Reclaim those dereferenced blocks!

**Q:** Why does my RBD images grow, but never shrink?

**A:** Thin provisioning — RADOS objects are created on write, and only freed by an **UNMAP** operation that travels the whole chain: **guest** → **hypervisor** → **RBD client**

- Linux guest: **fstrim -av** or **mount -o discard**
- Proxmox: **VirtIO SCSI** + **discard=on** + **ssd=1**.
  VMware: thin VMDK + VMFS6 + NVMe-oF **UNMAP**.
- krbd / librbd: **native UNMAP**, objects freed per 4 MiB.
  **rbd du** to measure, **rbd sparsify** if zeroed.



```
# Guest: check discard support, trim nightly
$ lsblk -D           # DISC-GRAN/DISC-MAX != 0
$ systemctl enable --now fstrim.timer
# Proxmox host
$ qm set 101 --scsihw virtio-scsi-single \
    --scsi0 ceph:vm-101-disk-0,discard=on,ssd=1
# Ceph: measure, then free zeroed ranges
$ rbd du rbd/vm-101-disk-0
$ rbd sparsify rbd/vm-101-disk-0
```

# Dedup on RBD? Think VDO (in-kernel)

**Ceph does no native dedup on RBD or RADOS**

Nothing at the block level today, but **VDO** can fill that gap.

**VDO: inline dedup + compression**

Bought from Permabit by Red Hat, mainlined in Linux 6.9: dedup, compression and elimination of zero blocks.

**Stack it on top of an RBD image**

Dedup domain local to the volume, RAM (UDS index) and CPU cost. Very effective, pleasant to use and well suited to redundant data.



```
$ rbd create --size 2T rbd/vdo-backing
$ rbd device map rbd/vdo-backing     #
/dev/rbd0
$ pvcreate /dev/rbd0
$ vgcreate vg_vdo /dev/rbd0
$ lvcreate --type vdo -n lv_vdo -l 100%FREE \
    -V 6T vg_vdo/vdopool
$ mkfs.xfs -K /dev/vg_vdo/lv_vdo
$ vdostats --human-readable    # saving %
```

SECTION 03

# What if I’m using CephFS?

File layouts to an EC pool, migrate existing data, reclaim space

# With CephFS, use file layouts!

**Map subfolders to EC**

File layouts: **new files land there directly**.

**Migrate existing data to reclaim space**

Move existing files to the EC pool with vcephfs_transcoder by Anthony D'Atri.

**Stay with replication for sensitive paths**

Keep small-IO, latency-sensitive directories on the replicated pool.



```
$ ceph fs add_data_pool cephfs cephfs-ec
$ setfattr -n ceph.dir.layout.pool -v cephfs-ec \
    /mnt/cephfs/archive
$ getfattr -n ceph.dir.layout /mnt/cephfs/archive
# Where does an existing file live?
$ getfattr -n ceph.file.layout.pool \
    /mnt/cephfs/archive/2024.tar
cephfs_data
```

SECTION 04

# S3: optimization, compression & deduplication

Compression, dynamic placement, optimized retention, deduplication



# S3: Dynamic Placement and Optimized Retention

**Dynamic Placement**

Land large objects in compressed EC pools the moment they are written, thanks to **Lua scripting** (criteria: size; bucket, tenant, etc.)

**Optimized Retention**

Automatically move objects to a compressed EC Storage Class with **Lifecycle Policies** (criteria: name, size, bucket)

**Automatic cleanup**

Object versions, incomplete multipart uploads parts, etc.



# S3: Compress at the RGW, not at the OSD

**Better compression ratio**

The RGW sees whole objects: larger datasets **compress much better** than BlueStore's per-blob compression.

**Compress once, ship less**

Compressed once at the RGW — not once per OSD — and shipped compressed: 2-3x more data written for the network **throughput** used.

**Faster recovery**

Generating fewer RADOS objects **speeds up recovery and backfilling**.



# S3: RGW compression in practice

### Enable it per Storage Class

- Deploy **multiple storage classes** in S3 with different algorithms — per workload, bucket, tenant or object size.

### Which algorithm?

- **zstd**: good ratio/speed trade-off
- **lz4**: faster, at the cost of lower ratios

### Hardware offloading is supported

- **Intel QAT**
- **UADK** on **ARM64** CPUs (HiSilicon Kunpeng 920) since Tentacle.



```
$ radosgw-admin zone placement modify \
    --rgw-zone default \
    --placement-id default-placement \
    --storage-class STANDARD \
    --compression zstd
$ ceph orch restart rgw.<service>
# size_utilized < size_actual = it works
$ radosgw-admin bucket stats --bucket=<bucket>
```

# S3: Compressing existing data

**Lifecycle policies for existing objects**

Transition objects to a compressed Storage Class — compressed as data moves (requires **PR #67725**).

**The full recipe**

- s3-dynamic-placement-and-archiving on GitHub
- Ceph Days India 2026 and FOSDEM 2026 talks.

github.com/frednass

# S3: deduplication

**Q:** Can Ceph finally deduplicate S3 objects?

**A:** Starting with Tentacle (v20), you can estimate the gains with **radosgw-admin dedup estimate** and **radosgw-admin dedup stats**

Deduplication itself will arrive in Umbrella (v21)

Managed per bucket, it will also apply to compressed and versioned objects.

PRs #62179, #68860, #68965




```
# Tentacle (v20): estimate only, nothing changed
$ radosgw-admin dedup estimate
$ radosgw-admin dedup stats
$ radosgw-admin dedup pause|resume|abort
# Umbrella (v21): dedup identical tail objects
$ radosgw-admin dedup estimate \
    --allow-bucket-list buckets.txt
$ radosgw-admin dedup exec --yes-i-really-mean-it
```

SECTION 05

# And at the RADOS layer?

EC, FastEC, BlueStore and RocksDB compression — and size=2: don’t!



# Erasure coding: pick the right workloads

**Q:** Which workloads fit EC, and which don't?

**A:** EC is ideal for large write-once/read-occasional data and high-throughput streaming.

GOOD FIT

## YES

Immutable S3 objects — photos, videos, backups — and streaming.



BAD FIT

## NO

High random IOPS — small files, databases on RBD and CephFS.



```
# 4+2 profile, ISA-L plugin, one chunk per host: 1.5x raw instead of 3x
$ ceph osd erasure-code-profile set ec42 plugin=isa technique=reed_sol_van k=4 m=2 crush-failure-domain=host
$ ceph osd pool create rbd-ec erasure ec42 --bulk
$ ceph osd pool set rbd-ec allow_ec_overwrites true      # required for RBD and CephFS data
$ ceph osd pool application enable rbd-ec rbd            # same recipe for cephfs-ec, RGW data pools
```

# On the horizon: FastEC in Tentacle

### EC gets much faster

FastEC targets **RBD and CephFS over EC**.

2×+ the IOPS of classic EC while keeping latencies low (**< 1.5 ms**) — but still half the IOPS of RF3.

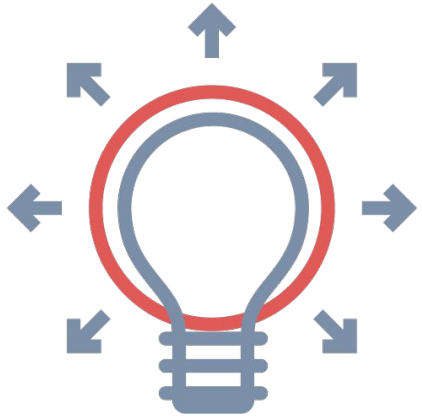
### Test it in a lab

- Start from an EC pool using the **ISA-L** plugin (jerasure is unmaintained) and a **16k stripe_unit**
- Reproduce a workload representative of your usage, and measure.

### Hold off a little longer…

As of October 2026, **do not enable** FastEC **in production**! See #80962 and #80963

```
# Lab only! All MONs and OSDs on Tentacle
$ ceph osd erasure-code-profile set fastec42 \
    plugin=isa technique=reed_sol_van \
    k=4 m=2 stripe_unit=16K \
    crush-failure-domain=host
$ ceph osd pool create lab-ec erasure fastec42
$ ceph osd pool set lab-ec allow_ec_overwrites true
$ ceph osd pool set lab-ec \
    allow_ec_optimizations true   # irreversible
```



# Performance: EC vs FastEC vs replication

*Source: **Lee Sanders**, "Fast Erasure Coding for Tentacle Performance Updates" on ceph.io*

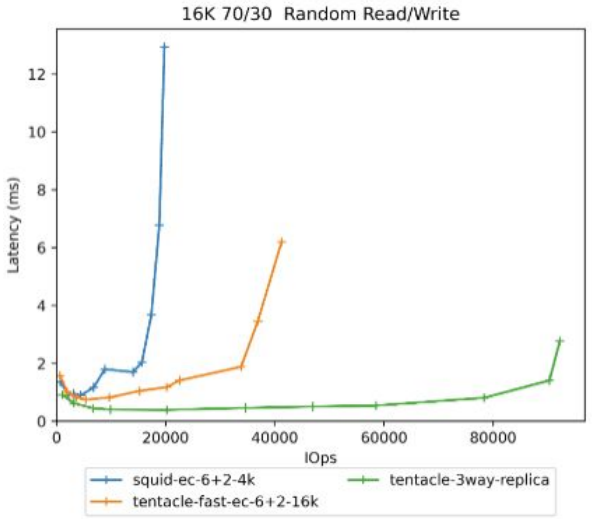
### Raw latency

- NVMe < **50 µs**
- HDD < **4-8 ms**

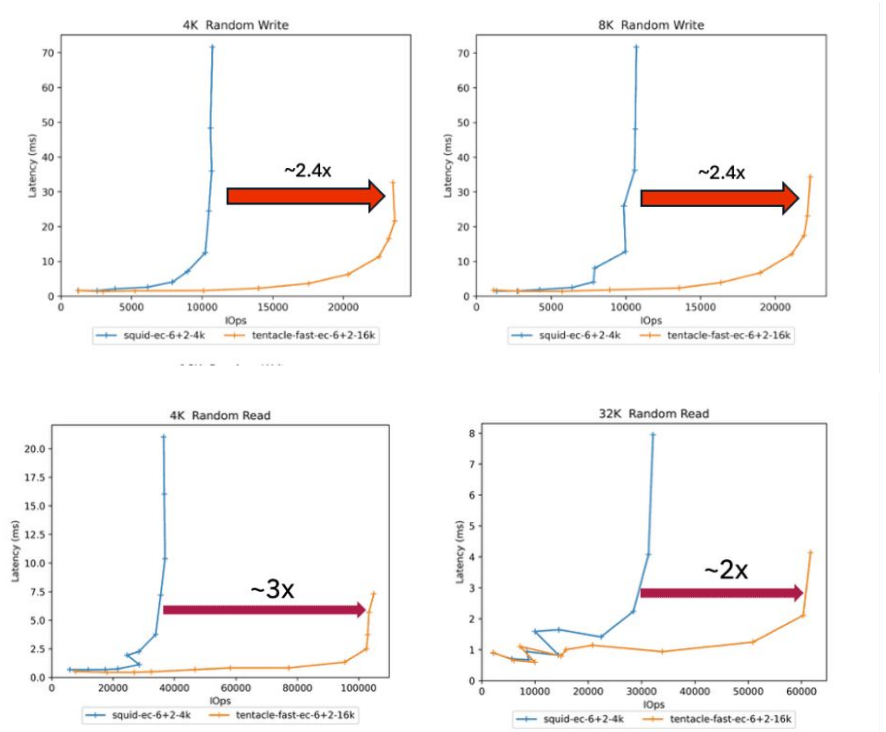
### Full-NVMe cluster, max latency for a single 4K IO

- RF3 (non-tuned) < **2.4 ms**
- RF3 (tuned) < **1.2 ms**
- EC4+2 without FastEC (tuned) < **3 ms**
- EC4+2 with FastEC (tuned) < **1.5 ms**
- EC4+2 with FastEC (tuned) + Crimson & Seastore < **~1.2 ms**

### Where do we stand vs replication?




### FastEC's gain on small IOs (4K random)



# BlueStore compression: turning it on right

**Enable it per pool, in aggressive mode**

**compression_mode aggressive** — safe, regardless of client hints.

**Poorly compressible data is not compressed**

**bluestore_compression_required_ratio** (0.875) discards poor results

**CPU overhead: measure it on your cluster**

Depends on compression mode, data compressibility, required ratios, IO sizes and flow patterns.



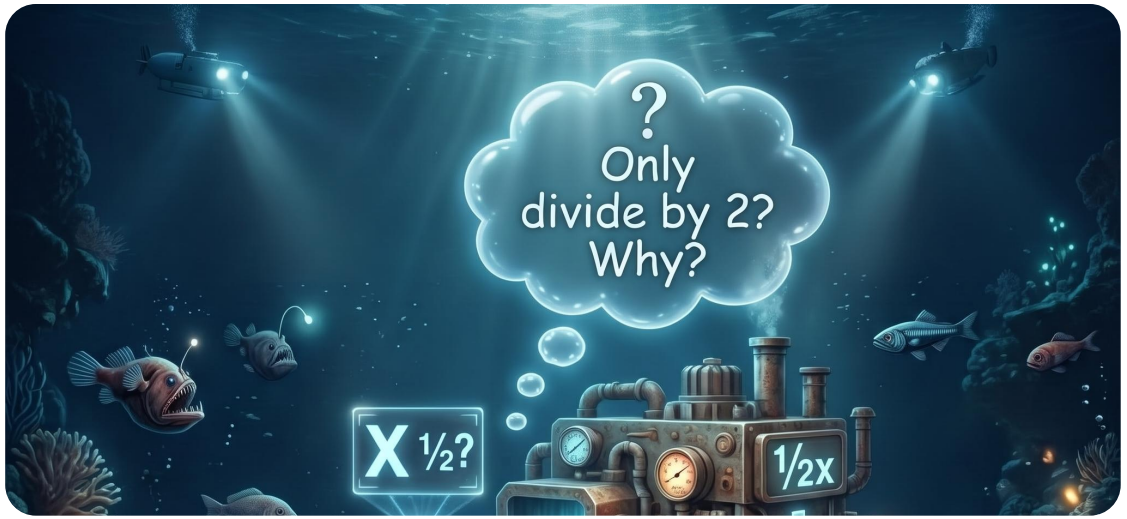
```
$ ceph osd pool set <pool> compression_mode \
    aggressive
$ ceph osd pool set <pool> compression_algorithm \
    snappy                          # or lz4, zstd
$ ceph osd pool set <pool> \
    compression_required_ratio 0.875
# Gains: USED COMPR vs UNDER COMPR columns
$ ceph df detail
```

# Getting past the (2×) compression ceiling

**Q:** Why does **ceph df detail** show a uniform ratio of **x2**?

**A:** **bluestore_compression_min_blob_size_hdd** defaults to 8k, which caps the ratio at **x2** even for **snappy** — see PR #67421.

- Raise it to **64K** and restart the OSDs for better ratios today.
- Soon, restarting the OSDs for this setting will no longer be needed (PR #67433).
- Compress existing data by forcing PG migration: **ceph osd out** then **ceph osd in**




```
$ ceph config set osd \
    bluestore_compression_min_blob_size_hdd 65536
$ ceph orch daemon restart osd.12   # one by one
# Rewrite old data, one OSD at a time
$ ceph osd out 12
$ ceph pg stat     # wait for all PGs
active+clean
$ ceph osd in 12
```

# RocksDB compression: turning it on!

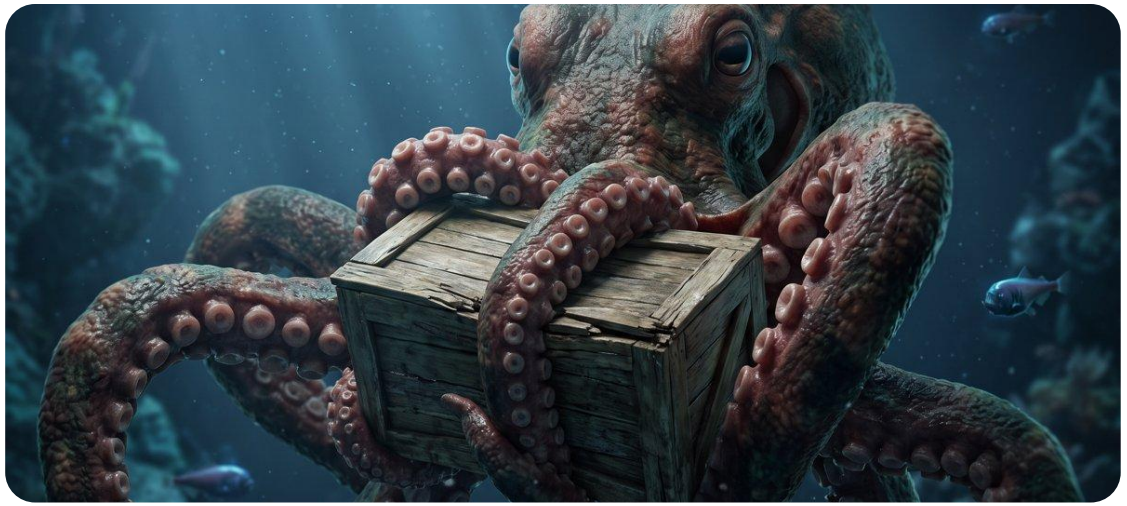
**Hybrid OSDs? Buy fewer NVMes…**

- Thanks to RocksDB compression, usable since Pacific, RocksDB databases have shrunk by **2.5 ×**

**The historical 4% DB/DATA ratio is now 2.5%**

- That lets you run **24 OSDs of 18 TB** on **3 NVMes of 3.84 TB** while honoring the **2.5%**

**Save on NVMes and reduce costs!**



```
$ ceph config set osd \
    bluestore_rocksdb_options_annex \
    compression=kLZ4Compression
$ ceph orch daemon restart osd.12   # one by
one
$ ceph tell osd.12 compact
# DB usage per OSD: META column
$ ceph osd df
```

# size = 2, min_size = 1: just say NO!

**Q:** Can we reclaim space by moving to size=2?

**A:** No. Experience shows a **high risk of data loss** with size = 2, min_size = 1:

- concurrent drive failures,
- degraded states caused by flapping OSDs,
- human error during maintenance.

```
# Spot risky replicated pools
$ ceph osd pool ls detail | grep 'size [12] min_size'
# Back to safe values
$ ceph osd pool set <pool> size 3
$ ceph osd pool set <pool> min_size 2
$ ceph config set global osd_pool_default_size 3
$ ceph config set global osd_pool_default_min_size 2
# EC pools: keep min_size = k+1
```



SECTION 06

# Hardware

OSD memory allocation, ZRAM, consumer drives



# Right-size OSD memory too

### Measure the onode cache hit ratio

Run **ceph tell osd.X perf dump** on OSDs with significant uptime; aim for an **onode_cache** hit ratio of **85–90%** or more.

### Don’t pay for unused RAM

- If the target ratio met with **osd_memory_target** = **8 GB**? No need to allocate 16 GB.
- Short on RAM? Try **ZRAM**

### Size future servers from real data

Cache metrics from production are the best input for sizing the next hardware purchase.

```
# ratio = onode_hits / (onode_hits +
onode_misses)
$ ceph tell osd.12 perf dump bluestore \
    | grep -E '"onode_(hits|misses)"'
$ ceph config set osd osd_memory_target 8G
# cephadm autotunes by default: pin it instead
$ ceph config set osd \
    osd_memory_target_autotune false
```

```
# ZRAM: compressed swap in RAM
$ zramctl --find --size 16G --algorithm zstd
$ mkswap /dev/zram0 && swapon -p 100 /dev/zram0
$ zramctl          # DATA vs COMPR = actual ratio
```

# Consumer-grade drives: very dangerous!

01

**Performance degradation**

**No sustained writes**, **shallow queues**: severe latency spikes and throttling under continuous load.

02

**No PLP protection (Power Loss Protection)**

Without PLP, **a power cut can corrupt or lose in-flight data** and undermine Ceph's consistency.

03

**Low write endurance**

Ceph's write amplification burns through low TBW ratings: **rapid wear-out**, **failures** and **more frequent replacements**.

# Key takeaways

**If you can’t add capacity…**

- **Extending the life of reliable hardware** past its support window can be considered
- Be selective: **compression** for large compressible data; **EC** where the IO pattern fits, and soon **deduplication**

**Never compromise the physical layer**

Enterprise-grade drives and size=3 (or EC)

**Reliability** and **durability** always come before cost optimization.

**Prepare for Tentacle & Umbrella improvements**

Test **FastEC** and **S3 dedup** on a representative sample of your data; not in production yet.



# Thank You!

Questions or thoughts? · Frédéric Nass · frederic.nass@clyso.com




---

Docs: **docs.ceph.com**

Mailing lists: **lists.ceph.io**

Clyso KB: **docs.clyso.com/docs/kb**

Community: **ceph.io**

Code: **github.com/ceph/ceph**